



CSAI 351: Final Project

Dr. Lobna Nassar

Spring – 2025

<i>STUDENT NAME</i>	<i>STUDENT ID</i>
Aya AlShalbe	2022005744
Habiba Eldib	2022005480
Jana Sayed	2022005444
Jannat Waqass	2022005413
Rabaa Alshbib	2022005690
Tasnim Alsaid	2022005779

Date: 27-04-2025

Table of Contents

1.	Introduction.....	3
2.	Literature Review	3
3.	Data Collection, Preprocessing & EDA	4
	• 3.1 Dataset Matching:	4
	• 3.2 Problem of Inconsistency & Clustering Solution:	4
	• 3.3 Dataset Cleaning & Preprocessing:	5
	• 3.4 Dataset Integration:	5
	• 3.5 Exploratory Data Analysis:	6
	• 3.6 Re-Clustering & Visualization Post Integration:	7
4.	Supervised Learning: Model Development	7
5.	Evaluation & Comparison with Literature	8
	• 5.1 K-Nearest Neighbor:	8
	• 5.2 Random Forest:	8
	• 5.3 Naïve Bayes Classification:	9
	• 5.4 Decision Tree Model:	9
	• 5.5 Comparison with Research Paper:	10
6.	NLP Component.....	10
7.	Team Contributions.....	13
8.	Conclusion	14
9.	References	14

1. Introduction

This project focuses on predicting air quality levels using machine learning models by integrating and analysing real-world datasets. Inspired by the "AirNet" study, we merged pre-labelled and unlabelled datasets through clustering and standardization to create a unified dataset. Various classification models—K-Nearest Neighbour, Random Forest, Decision Tree, and Naïve Bayes—were then developed, evaluated, and compared to identify the most effective approach for air quality forecasting.

2. Literature Review

Air pollution poses a critical threat to public health and the environment, contributing to respiratory illnesses, cardiovascular diseases, and premature deaths globally. According to the World Health Organization, nearly 7 million people die annually due to air pollution-related complications, highlighting the urgent need for accurate and timely air quality forecasting systems (Tipton, 2022). In recent years, predictive models using machine learning (ML) and deep learning (DL) have gained significant attention as they offer promising solutions for real-time environmental monitoring.

Numerous studies have shown high accuracy using ML and DL for predicting air quality indicators like PM_{2.5}, NO₂, CO, and O₃:

- **Kumar and Pande (2023):** Used classifiers (Naïve Bayes, Random Forest, KNN, SVM), achieving **up to 90% accuracy**.
- **2- Sanjeev (2021):** Used Random Forest, SVM, ANN with features like CO, CH₄, NO_x, O₃, humidity; achieved **97% accuracy**.
- **3- Jin et al. (2023):** Proposed a **graph neural network model**, achieving **99% accuracy** on PM_{2.5} forecasting.
- **4-Basel et al. (2020):** Used an **LSTM-based model** for climate air quality forecasting, achieving **100% accuracy**.

Other studies developed specialized models, with Abirami and Chitra (2023) improving accuracy by 30–37% using STAA-LSTM, and Kapoor et al. (2023) achieving up to 98% accuracy in indoor CO₂ prediction using ML algorithms like ANN, GPR, SVM, and ensemble methods. Rahman et al. (2024) introduced AirNet, a Django-based web app that predicted air quality using a Kaggle dataset with 23,463 records, where Random Forest and Decision Tree models achieved 100% accuracy. Although AirNet achieved high accuracy, it lacks deep learning models, real-time sensor integration, and excludes geographical data, limiting its adaptability.

In summary, machine learning and deep learning models have proven highly effective for air quality forecasting, achieving high accuracy. The AirNet system adds to this progress with a scalable, user-friendly platform, though future work should focus on integrating real-time data, expanding to diverse climates, and using hybrid or deep learning models for further improvement.

3. Data Collection, Preprocessing & EDA

- 3.1 Dataset Matching:

A real-world dataset (dataset1.csv) was found on Kaggle with features that closely matched the ones used in the AirNet study. The dataset included:

- AQI Value
- CO AQI Value
- Ozone AQI Value
- NO2 AQI Value
- PM2.5 AQI Value

What made this dataset suitable was that it already contained labelled AQI categories (e.g., Good, Moderate, Poor, Hazardous), allowing for direct supervised comparison.

```
RangeIndex: 23463 entries, 0 to 23462
Data columns (total 12 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Country                23036 non-null  object
1   City                   23462 non-null  object
2   AQI Value              23463 non-null  int64
3   AQI Category           23463 non-null  object
4   CO AQI Value           23463 non-null  int64
5   CO AQI Category        23463 non-null  object
6   Ozone AQI Value        23463 non-null  int64
7   Ozone AQI Category     23463 non-null  object
8   NO2 AQI Value          23463 non-null  int64
9   NO2 AQI Category       23463 non-null  object
10  PM2.5 AQI Value        23463 non-null  int64
11  PM2.5 AQI Category     23463 non-null  object
dtypes: int64(5), object(7)
```

Fig1. Information about columns in dataset1 (research paper dataset)

```
RangeIndex: 5000 entries, 0 to 4999
Data columns (total 10 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Temperature                           5000 non-null   float64
1   Humidity                              5000 non-null   float64
2   PM2.5                                 5000 non-null   float64
3   PM10                                  5000 non-null   float64
4   NO2                                    5000 non-null   float64
5   SO2                                    5000 non-null   float64
6   CO                                     5000 non-null   float64
7   Proximity_to_Industrial_Areas         5000 non-null   float64
8   Population_Density                    5000 non-null   int64
9   Air Quality                           5000 non-null   object
dtypes: float64(8), int64(1), object(1)
```

Fig2. Information about columns in dataset2 (our dataset)

- 3.2 Problem of Inconsistency & Clustering Solution:

Since the AirNet dataset was initially unclassified and our Kaggle dataset was already labelled, a challenge arose in merging the two. To overcome this, we applied **K-Means clustering** to the unlabelled dataset (dataset1) to:

- Identify patterns in pollution metrics
- Classify the data into four clusters that align with AQI categories
- Assign labels (Good, Moderate, Poor, Hazardous) to those clusters based on their pollutant levels

This made the data compatible with our classified dataset for integration.

Cluster Centroids:									
	AQI Value	CO	AQI Value	Ozone	AQI Value	NO2	AQI Value	PM2.5	AQI Value
0	181.626163		1.566701		41.969493		1.834540		177.848501
1	51.444100		0.995109		30.224495		2.004628		48.397928
2	124.889189		4.563514		14.922297		18.545946		124.844595
3	169.631171		3.290416		143.027106		2.670862		153.547919

Fig3. Calculation of Cluster Centroids using K-Means Clustering

```

RangeIndex: 23463 entries, 0 to 23462
Data columns (total 14 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Country                               23036 non-null  object
1   City                                  23462 non-null  object
2   AQI Value                             23463 non-null  int64
3   AQI Category                           23463 non-null  object
4   CO AQI Value                           23463 non-null  int64
5   CO AQI Category                         23463 non-null  object
6   Ozone AQI Value                         23463 non-null  int64
7   Ozone AQI Category                     23463 non-null  object
8   NO2 AQI Value                           23463 non-null  int64
9   NO2 AQI Category                       23463 non-null  object
10  PM2.5 AQI Value                         23463 non-null  int64
11  PM2.5 AQI Category                     23463 non-null  object
12  Cluster                                23463 non-null  int64
13  Predicted_AQI_Category                 23463 non-null  object
dtypes: int64(6), object(8)

```

Fig4. Information about columns of dataset1 after clustering

• 3.3 Dataset Cleaning & Preprocessing:

Before merging the two datasets, the following preprocessing steps were done:

- **Removal of missing values** to ensure data quality
- **Outlier removal** using the Interquartile Range (IQR) method to handle anomalies in pollutant readings
- **Column renaming** for consistency (e.g., converting 'CO AQI Value' to 'CO')

• 3.4 Dataset Integration:

After clustering and standardizing column names, we concatenated both datasets into a single unified Data Frame. The features included:

- CO
- NO2
- PM2.5

- Air Quality (Label)

This provided a larger, more robust dataset for downstream analysis.

- **3.5 Exploratory Data Analysis:**

Extensive EDA was conducted to understand feature distributions and interdependencies:

Pair plot Analysis

- Visualized feature relationships across different AQI labels
- Helped identify potential overlapping patterns or clear boundaries between categories

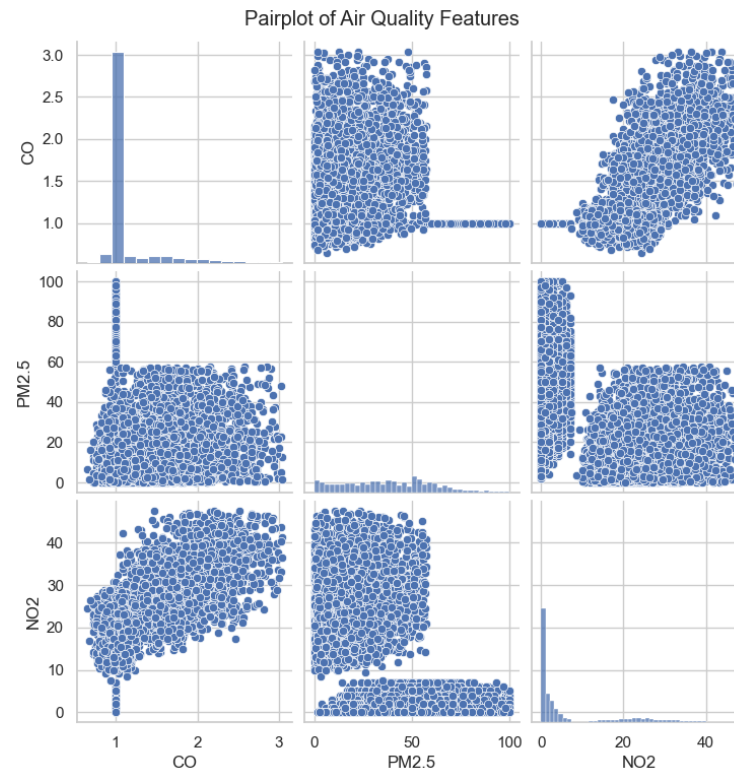


Fig5. Pair plot for integrated dataset

PCA (Principal Component Analysis)

- Reduced dimensionality from 3D (CO, PM2.5, NO2) to 2D for visualizing cluster structure
- Helped confirm that natural groupings existed within the merged dataset



Fig6. PCA of Air Quality Data for integrated dataset.

- **3.6 Re-Clustering & Visualization Post Integration:**

After combining the datasets and applying PCA, we re-applied K-Means clustering to:

- Explore if new clusters emerged in the combined feature space
- Evaluate the structure and validity of prior labels

Scatter plots of PCA components coloured by cluster were generated to visualize and verify this step.

The final output of the project was a clean, well-labelled dataset titled `integrated_air_quality.csv`, which includes normalized pollutant values and corresponding AQI categories.

This dataset is now suitable for supervised learning, real-time dashboarding, or longitudinal trend analysis.

4. Supervised Learning: Model Development

After integrating the dataset of the research paper with the one we looked for, based on their common attributes and the class label, we can immediately start the classification on the integrated dataset.

At this stage, we used **4 classification models** for the sake of comparison to see which one of them did the best job through some of the classification measurements, including Accuracy, Recall, Precision and F1score which combines both precision and recall in a single formula.

3 of the deployed classifiers are the same as the classifiers in the research paper, which are **KNN**, **Decision Tree**, and **Random Forest**, and 1 is different, which is the **Naïve Bayes classifier**. The implementation of all these classifiers is almost the same since we are separating the features from the class label at the first place then spreading the training dataset from the testing one, then passing the training data to the model to train itself, and then passing the testing records, which were hidden at the first place from the model to prevent it from learning the data itself. In addition, 30% of the records will be used for testing and 70% for training as shown below:

```
# Separate features and labels for all classifiers

x = final_df[['CO', 'PM2.5', 'NO2']]
y = final_df['Air Quality']

# Split the data into train and test sets (30% test) for all the classifiers

x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.3, random_state=42)
```

5. Evaluation & Comparison with Literature

• 5.1 K-Nearest Neighbor:

There are struggles in “Hazardous” and “Poor” attributes due to imbalanced data where Hazardous samples are 83 and Moderate samples are 4169. KNN perform better in small datasets, and KNN’s nearest neighbours’ logic is easier to explain ad-hoc. It’s sensitive to outliers. It gives lower accuracy (87%).

***** KNN Classification Report *****				
	precision	recall	f1-score	support
Good	0.97	0.99	0.98	612
Hazardous	0.72	0.53	0.61	83
Moderate	0.99	0.99	0.99	4169
Poor	0.72	0.72	0.72	246
accuracy			0.97	5110
macro avg	0.85	0.81	0.83	5110
weighted avg	0.97	0.97	0.97	5110

• 5.2 Random Forest:

It consistently outperforms KNN, especially for Hazardous (higher precision) and Poor (better recall). Handles class imbalance better due to ensemble learning (grouping learning). Random Forest is unbiased towards major of classes, and it’s less sensitive to noise. As a result, RF model is adopted because it gives priority to accuracy (97%) which is higher than KNN accuracy (87%) and better minority class performance (hazardous and poor).

***** Random Forest Classification Report *****				
	precision	recall	f1-score	support
Good	0.98	0.98	0.98	612
Hazardous	0.74	0.55	0.63	83
Moderate	0.99	0.99	0.99	4169
Poor	0.74	0.75	0.74	246
accuracy			0.97	5110
macro avg	0.86	0.82	0.84	5110
weighted avg	0.97	0.97	0.97	5110

<u>Metric</u>	<u>KNN</u>	<u>Random Forest</u>	<u>Winner</u>	<u>Why?</u>
Macro F1	0.83	0.84	RF	Better balance across all classes.
Hazardous F1	0.61	0.63	RF	Fewer missed hazardous cases.
Poor Recall	0.72	0.75	RF	Both excel for the majority class.
Moderate F1	0.99	0.99	Tie	Both excel for the majority class.

If we compare Random Forest (RF) with K-nearest neighbours (KNN), RF's recall is marginally lower on average than KNNs but wins in the "Hazardous" and "Poor" classes, which are often high-risk and high-stakes. Despite KNN's overall HR-AUC, recall on "Hazardous" was lower than 0.60 (0.53, specifically). KNN's recall for "Hazardous" suggests risk if a warning should have been provided (missed recalls).

- **5.3 Naïve Bayes Classification:**

Naïve Bayes classification demonstrated the lowest accuracy among all the models evaluated, primarily due to its assumption of feature independence. This model treats each attribute separately, without considering the interactions between them. However, in our air quality dataset, attributes like temperature and humidity are interrelated and jointly influence pollution levels. Ignoring these relationships limits Naïve Bayes' ability to accurately model the data.

Additionally, Naïve Bayes tends to favour majority classes, such as 'Moderate.' As a result, it achieved relatively high precision, recall, and F1-score for the 'Moderate' class. However, its performance for minority classes like 'Hazardous' and 'Poor' was significantly weaker and imbalanced.

***** Naive Bayes Classification Report *****				
	precision	recall	f1-score	support
Good	0.93	0.87	0.90	612
Hazardous	0.76	0.53	0.62	83
Moderate	0.98	0.91	0.95	4169
Poor	0.39	0.93	0.55	246
accuracy			0.90	5110
macro avg	0.76	0.81	0.76	5110
weighted avg	0.94	0.90	0.92	5110

We also observed that the precision, recall, and F1-score varied considerably even within the same classes. Since Naïve Bayes relies on probability estimates that are sensitive to feature distribution, its predictions can fluctuate notably with different data patterns. Moreover, by relying solely on basic probability without considering combinations of attributes, Naïve Bayes sometimes achieved high recall but very low precision for rare classes, indicating a tendency to overpredict these classes and produce more errors.

Overall, these limitations make Naïve Bayes less suitable for complex and imbalanced classification problems like air quality prediction.

- **5.4 Decision Tree Model:**

For the Decision Tree classification, the model achieved a strong accuracy of 96%. This success can be attributed to its ability to learn from the dataset and create structured decision rules based on the most relevant features. The model splits the data into branches by applying logical conditions, such as "Is the humidity above a certain value?" or "Is the temperature below a specific threshold?"—guiding the model toward accurate predictions.

In our dataset, where variables like temperature and humidity interact and jointly influence air quality, the Decision Tree effectively captured these relationships. It developed clear decision paths that reflected how different combinations of these features corresponded to various pollution levels.

***** Decision Tree (Entropy) Classification Report *****				
	precision	recall	f1-score	support
Good	0.97	0.97	0.97	612
Hazardous	0.54	0.54	0.54	83
Moderate	0.98	0.98	0.98	4169
Poor	0.62	0.63	0.63	246
accuracy			0.96	5110
macro avg	0.78	0.78	0.78	5110
weighted avg	0.96	0.96	0.96	5110

Moreover, precision, recall, and F1-score remained consistent across all classes. This consistency is due to the model's straightforward if-then decision rules, which promote stable and reliable outcomes across different pollution categories.

- **5.5 Comparison with Research Paper:**

<u>Research paper</u>	<u>Integrated dataset</u>
<i>Dataset size: +23000</i>	<i>Dataset size: +17000</i>
<i>More stable results</i>	<i>Less stable results</i>
<i>Higher accuracy in KNN and RF</i>	<i>Less accuracy in KNN and RF</i>
<i>hard classes were handled better</i>	<i>struggled with hazardous air quality</i>
<i>RF achieved perfect classification 100%.</i>	<i>RF makes small mistakes</i>
<i>Random Forest beats KNN</i>	<i>Random Forest beats KNN</i>
<i>cleaner preprocessing</i>	<i>class imbalance</i>
<i>Optimized models</i>	<i>Real-word cases</i>
<i>Decision tree achieves 96%</i>	<i>Decision tree achieves 100% in accuracy</i>

6. NLP Component

NLP Classification Task ¶

Let's move on to the actual task. To make things a little easier, go ahead and only grab reviews that were either 1 star or 5 stars.

Create a dataframe called `yelp_class` that contains the columns of `yelp` dataframe but for only the 1 or 5 star reviews.

The Data

Read the `YelpReview.csv` file and set it as a dataframe called `yelp`.

```
In [34]: yelp=pd.read_csv(r"C:\Users\JANA\Downloads\YelpReviews.csv")
```

```
In [16]: yelp_class = yelp[(yelp['stars'] == 1)|(yelp['stars'] == 5)]
```

**** Create two objects X and y. X will be the 'text' column of `yelp_class` and y will be the 'stars' column of `yelp_class`. (Your features and target/labels)****

```
In [17]: # Assuming you've already loaded the DataFrame as `yelp`
x = yelp['text'] # Feature: text reviews
y = yelp['stars'] # Target: star ratings
```

Import CountVectorizer and create a CountVectorizer object. How CountVectorizer Works:

Tokenization: CountVectorizer splits the input text into individual words or tokens. By default, it tokenizes based on whitespace and removes punctuation.

Vocabulary Building: It builds a vocabulary of all the unique words (tokens) across the entire corpus of documents.

Feature Extraction: It creates a vector for each document, where each element represents the count of a specific word (based on the vocabulary) in that document.**

```
In [35]: from sklearn.feature_extraction.text import CountVectorizer

# Create a CountVectorizer object
cv = CountVectorizer()
# Fit the CountVectorizer to the text data and transform it into a document-term matrix
# Drop rows where the text is NaN
yelp = yelp.dropna(subset=['text'])

# Recreate X and y
X = yelp['text']
y = yelp['stars']

# Then try vectorizing again
X_vectorized = cv.fit_transform(X)
# Get the vocabulary (word -> column index)
vocab = cv.vocabulary_
len(vocab)
```

Out[35]: 29185

Feature Extraction

```
In [36]: # View as array (optional for small samples)
X_vectorized.toarray() # Caution: might be large!

# Example: show feature names
cv.get_feature_names_out()[:10] # Show first 10 words in vocabulary
```

Out[36]: array(['00', '000', '007', '00a', '00am', '00pm', '01', '02', '03',
 '03342'], dtype=object)

** Use the fit_transform method on the CountVectorizer object and pass in X (the 'text' column). Save this result by overwriting X.**

```
In [37]: # Overwrite X with the vectorized version
X = cv.fit_transform(X)
```

Train Test Split

Let's split our data into training and testing data.

** Use train_test_split to split up the data into X_train, X_test, y_train, y_test. Use test_size=0.3 and random_state=101 **

```
In [38]: from sklearn.model_selection import train_test_split

# Split the data: 70% train, 30% test
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=101)
```

Training a Model

Time to train a model!

Import MultinomialNB and create an instance of the estimator and call is nb

```
In [39]: from sklearn.naive_bayes import MultinomialNB  
  
# Create an instance of the model  
nb = MultinomialNB()
```

Now fit NB using the training data.

```
In [40]: # Train the model using the training data  
nb.fit(X_train, y_train)
```

```
Out[40]: ▾ MultinomialNB  
MultinomialNB()
```

Predictions and Evaluations

Time to see how our model did!

Use the predict method off of nb to predict labels from X_test.

```
In [41]: # Predict Labels (star ratings) for the test data  
y_pred = nb.predict(X_test)
```

Predictions and Evaluations

Time to see how our model did!

Use the predict method off of nb to predict labels from X_test.

```
In [41]: # Predict Labels (star ratings) for the test data  
y_pred = nb.predict(X_test)
```

Create a confusion matrix and classification report using these predictions and y_test

```
In [26]: from sklearn.metrics import confusion_matrix, classification_report
```

```
In [42]: # Create the confusion matrix  
cm = confusion_matrix(y_test, y_pred)
```

```
# Display the confusion matrix  
print("Confusion Matrix:")  
print(cm)
```

```
Confusion Matrix:  
[[ 57  35  12  79  37]  
 [ 12  20  33 180  28]  
 [  6   8  36 350  43]  
 [  6   4  21 775 258]  
 [  5   5   9 446 535]]
```

```
In [43]: # Generate the classification report
cr = classification_report(y_test, y_pred)

# Display the classification report
print("Classification Report:")
print(cr)
```

```
Classification Report:
              precision    recall  f1-score   support

     1.0         0.66       0.26       0.37         220
     2.0         0.28       0.07       0.12         273
     3.0         0.32       0.08       0.13         443
     4.0         0.42       0.73       0.54        1064
     5.0         0.59       0.54       0.56        1000

 accuracy                   0.47         3000
 macro avg                 0.46       0.34       0.34         3000
 weighted avg              0.47       0.47       0.43         3000
```

Verify the output of five test records

```
In [44]: # Display the first five true labels and predicted labels
print("True Labels: ", y_test.head(5).values)
print("Predicted Labels: ", y_pred[:5])
```

```
True Labels: [3. 4. 5. 5. 4.]
Predicted Labels: [4. 4. 5. 5. 4.]
```

Type *Markdown* and LaTeX: α^2

7. Team Contributions

Student Name	Contribution
Aya AlShalbe	Naive bayes code including its evaluation code and its confusion matrix + Decision tree code including its evaluation code and its confusion matrix
Habiba Eldib	Evaluate and analyze Naive bayes and Decision Tree; Compare Decision Tree in our dataset with the Decision Tree in the research paper
Jana Sayed	Literature Review and NLP code
Jannat Waqass	Searched and selected research papers & datasets; Performed K-means clustering on research dataset; Integration of the 2 datasets; EDA and Preprocessing
Rabaa Alshbib	Analyze and evaluate KNN and Random Forest models of the integrated dataset and compare them with the research paper
Tasnim Alsaid	KNN code including its evaluation code and its confusion matrix + Random Forest code including its evaluation code and its confusion matrix + merging all codes together

8. Conclusion

Through extensive data preprocessing, integration, and model evaluation, our project demonstrated that **Random Forest** and **Decision Tree models** achieved the highest accuracies, with Decision Tree reaching **96%**. While Naïve Bayes struggled due to its assumption of feature independence, the ensemble and rule-based models effectively captured complex relationships between pollutants. Overall, this project highlights the importance of thoughtful data preparation and model selection in building reliable air quality prediction systems.

9. References

1. Research Paper Dataset:

<https://www.kaggle.com/datasets/hasibalmuzdadid/global-air-pollution-dataset>

2. Our Dataset:

<https://www.kaggle.com/datasets/mujtabamatin/air-quality-and-pollution-assessment>

3. Research Paper:

Rahman, M. M., Nayeem, M. E. H., Ahmed, M. S., Tanha, K. A., Sakib, M. S. A., Uddin, K. M. M., & Babu, H. M. H. (2024). *AirNet: Predictive machine learning model for air quality forecasting using web interface*. Environmental Systems Research, 13(44).

<https://doi.org/10.1186/s40068-024-00378-z>